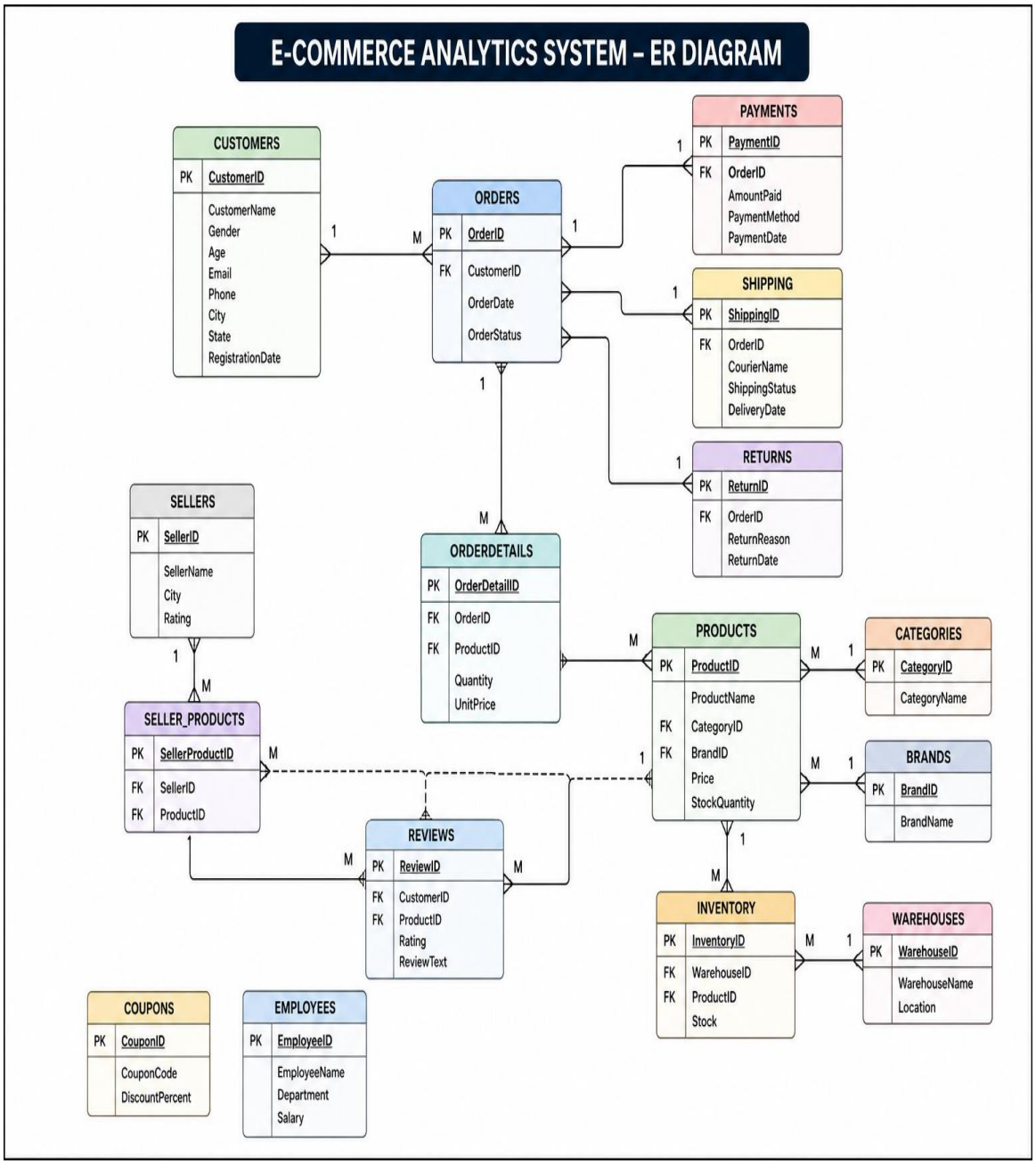
E-COMMERCE ANALYTICS SYSTEM

ID: 148239

NAME: Pooja S

GitHub URL: https://github.com/pooja-shindhe/E_COMMERCE_ANALYTICS_SYSTEM.git

Table Schema:



BASIC LEVEL

1. Display all customer details.

```
SELECT *
FROM customers;
```

---

2. Find customers above 60 years of age.

```
SELECT *
FROM customers
WHERE Age > 60;
```

---

3. Find customers whose name starts with 'A'.

```
SELECT *
FROM customers
WHERE CustomerName LIKE 'A%';
```

---

4. Find products priced between ₹10,000 and ₹50,000.

```
SELECT *
FROM products
WHERE Price BETWEEN 10000 AND 50000;
```

---

5. Display orders with status 'Delivered' or 'Shipped'.

```
SELECT *
FROM orders
WHERE OrderStatus IN ('Delivered','Shipped');
```

---

6. Display distinct customer cities.

```
SELECT DISTINCT City
FROM customers;
```

---

7. Display the top 10 highest-priced products.

```
SELECT *
FROM products
ORDER BY Price DESC
LIMIT 10;
```

---

8. Find the total number of customers in each city.

```
SELECT
   City,
   COUNT(*) AS TotalCustomers
FROM customers
GROUP BY City;
```

---

9. Find the total revenue received.

```
SELECT
   SUM(AmountPaid) AS TotalRevenue
FROM payments;
```

---

10. Find the average product price.

```
SELECT
   AVG(Price) AS AveragePrice
FROM products;
```

---

11. Find the highest and lowest product prices.

```
SELECT
   MAX(Price) AS HighestPrice,
   MIN(Price) AS LowestPrice
FROM products;
```

---

12. Find customers registered this year.

```
SELECT *
FROM customers
WHERE YEAR(RegistrationDate) = YEAR(CURDATE());
```

---

13. Find customers who have not placed any orders.

```
SELECT *
FROM customers
WHERE CustomerID NOT IN
(
   SELECT CustomerID
   FROM orders
);
```

---

14. Find the number of products available in each category.

```
SELECT
   CategoryID,
```

```
    COUNT(*) AS TotalProducts
FROM products
GROUP BY CategoryID;
```

---

15. Find the total available stock across all warehouses.

```
SELECT
    SUM(Stock) AS TotalStock
FROM inventory;
```

INTERMEDIATE LEVEL

16. Display customer names along with their order details.

```
SELECT
   c.CustomerID,
   c.CustomerName,
   o.OrderID,
   o.OrderDate,
   o.OrderStatus
FROM customers c
INNER JOIN orders o
ON c.CustomerID = o.CustomerID;
```

Concept: INNER JOIN

---

17. Display product names along with their category names.

```
SELECT
   p.ProductName,
   c.CategoryName
FROM products p
INNER JOIN categories c
ON p.CategoryID = c.CategoryID;
```

Concept: INNER JOIN

---

18. Display product names along with their brand names.

```
SELECT
   p.ProductName,
   b.BrandName
FROM products p
INNER JOIN brands b
ON p.BrandID = b.BrandID;
```

Concept: INNER JOIN

---

19. Find the total revenue generated by each customer.

```
SELECT
   c.CustomerID,
   c.CustomerName,
   SUM(p.AmountPaid) AS TotalRevenue
FROM customers c
INNER JOIN orders o
ON c.CustomerID = o.CustomerID
```

```
INNER JOIN payments p
ON o.OrderID = p.OrderID
GROUP BY
   c.CustomerID,
   c.CustomerName;
```

Concept: INNER JOIN, GROUP BY, SUM()

---

20. Find products that have never been sold.

```
SELECT
   p.ProductID,
   p.ProductName
FROM products p
LEFT JOIN orderdetails od
ON p.ProductID = od.ProductID
WHERE od.ProductID IS NULL;
```

Concept: LEFT JOIN, IS NULL

---

21. Find customers who placed more than one order.

```
SELECT
   CustomerID,
   COUNT(OrderID) AS TotalOrders
FROM orders
GROUP BY CustomerID
HAVING COUNT(OrderID) > 1;
```

Concept: HAVING

---

22. Find the total sales for each category.

```
SELECT
   c.CategoryName,
   SUM(od.Quantity * od.UnitPrice) AS TotalSales
FROM categories c
INNER JOIN products p
ON c.CategoryID = p.CategoryID
INNER JOIN orderdetails od
ON p.ProductID = od.ProductID
GROUP BY c.CategoryName;
```

Concept: INNER JOIN, GROUP BY

---

23. Display monthly sales.

```
SELECT
   YEAR(PaymentDate) AS Year,
   MONTH(PaymentDate) AS Month,
   SUM(AmountPaid) AS Revenue
FROM payments
GROUP BY
   YEAR(PaymentDate),
   MONTH(PaymentDate)
ORDER BY Year, Month;
```

Concept: GROUP BY, YEAR(), MONTH()

---

24. Display customer and employee names.

```
SELECT CustomerName AS Name
FROM customers

UNION

SELECT EmployeeName
FROM employees;
```

Concept: UNION

---

25. Display customer and employee names including duplicates.

```
SELECT CustomerName AS Name
FROM customers

UNION ALL

SELECT EmployeeName
FROM employees;
```

Concept: UNION ALL

---

26. Find customers who have placed at least one order.

```
SELECT *
FROM customers c
WHERE EXISTS
(
   SELECT 1
   FROM orders o
```

```
    WHERE o.CustomerID = c.CustomerID
);
```

Concept: EXISTS

---

27. Find customers who have never placed an order.

```
SELECT *
FROM customers c
WHERE NOT EXISTS
(
    SELECT 1
    FROM orders o
    WHERE o.CustomerID = c.CustomerID
);
```

Concept: NOT EXISTS

---

28. Find products frequently purchased together.

```
SELECT
    od1.ProductID AS ProductA,
    od2.ProductID AS ProductB,
    COUNT(*) AS PurchaseCount
FROM orderdetails od1
INNER JOIN orderdetails od2
ON od1.OrderID = od2.OrderID
AND od1.ProductID < od2.ProductID
GROUP BY
    od1.ProductID,
    od2.ProductID;
```

Concept: SELF JOIN

---

29. Create a view to display customer revenue.

```
CREATE VIEW vw_customer_revenue AS

SELECT
    c.CustomerID,
    c.CustomerName,
    SUM(p.AmountPaid) AS Revenue
FROM customers c
INNER JOIN orders o
ON c.CustomerID = o.CustomerID
INNER JOIN payments p
ON o.OrderID = p.OrderID
```

```
GROUP BY
   c.CustomerID,
   c.CustomerName;
```

Concept: VIEW

---

30. Create a view to display product sales.

```
CREATE VIEW vw_product_sales AS

SELECT
   p.ProductID,
   p.ProductName,
   SUM(od.Quantity * od.UnitPrice) AS Revenue
FROM products p
INNER JOIN orderdetails od
ON p.ProductID = od.ProductID
GROUP BY
   p.ProductID,
   p.ProductName;
```

Concept: VIEW

ADVANCED SQL (STORED PROCEDURE, TRIGGERS)

31. Find Products Costing More Than the Average Price

```
SELECT *
FROM products
WHERE Price >
(
   SELECT AVG(Price)
   FROM products
);
```

Concept: Subquery

---

32. Find Customers Who Spent More Than the Average Customer Spending

```
SELECT
   c.CustomerID,
   c.CustomerName,
   SUM(p.AmountPaid) AS TotalSpent
FROM customers c
INNER JOIN orders o
ON c.CustomerID = o.CustomerID
INNER JOIN payments p
ON o.OrderID = p.OrderID
GROUP BY
   c.CustomerID,
   c.CustomerName
HAVING SUM(p.AmountPaid) >
(
   SELECT AVG(CustomerSpend)
   FROM
   (
      SELECT
         SUM(AmountPaid) AS CustomerSpend
      FROM payments
      GROUP BY OrderID
   ) AS AvgSpend
);
```

Concept: Nested Subquery

---

33. Find Top 5 Customers by Revenue Using CTE

```
WITH CustomerRevenue AS
(
   SELECT
      c.CustomerID,
      c.CustomerName,
      SUM(p.AmountPaid) AS Revenue
```

```
    FROM customers c
    INNER JOIN orders o
        ON c.CustomerID = o.CustomerID
    INNER JOIN payments p
        ON o.OrderID = p.OrderID
    GROUP BY
        c.CustomerID,
        c.CustomerName
)

SELECT *
FROM CustomerRevenue
ORDER BY Revenue DESC
LIMIT 5;
```

Concept: CTE

---

34. Rank Products Based on Revenue

```
SELECT
    ProductID,
    SUM(Quantity * UnitPrice) AS Revenue,
    RANK() OVER
    (
        ORDER BY SUM(Quantity * UnitPrice) DESC
    ) AS ProductRank
FROM orderdetails
GROUP BY ProductID;
```

Concept: Window Function, RANK()

---

35. Find the Second Highest Spending Customer

```
SELECT *
FROM
(
    SELECT
        c.CustomerID,
        c.CustomerName,
        SUM(p.AmountPaid) AS TotalSpent,
        DENSE_RANK() OVER
        (
            ORDER BY SUM(p.AmountPaid) DESC
        ) AS RankNo
    FROM customers c
    INNER JOIN orders o
        ON c.CustomerID = o.CustomerID
    INNER JOIN payments p
        ON o.OrderID = p.OrderID
    GROUP BY
```

```
        c.CustomerID,
        c.CustomerName
) x
WHERE RankNo = 2;
```

Concept: DENSE_RANK()

---

36. Display Running Revenue Month-wise

```
SELECT
   YEAR(PaymentDate) AS YearNo,
   MONTH(PaymentDate) AS MonthNo,
   SUM(AmountPaid) AS Revenue,

   SUM(SUM(AmountPaid))
   OVER
   (
      ORDER BY YEAR(PaymentDate),
            MONTH(PaymentDate)
   ) AS RunningRevenue

FROM payments

GROUP BY
YEAR(PaymentDate),
MONTH(PaymentDate);
```

Concept: SUM() OVER()

---

37. Rank Products Within Each Category

```
SELECT

   p.CategoryID,

   p.ProductID,

   SUM(od.Quantity * od.UnitPrice) AS Revenue,

   RANK() OVER
   (
      PARTITION BY p.CategoryID
      ORDER BY SUM(od.Quantity * od.UnitPrice) DESC
   ) AS CategoryRank

FROM products p

INNER JOIN orderdetails od
ON p.ProductID = od.ProductID
```

```
GROUP BY
p.CategoryID,
p.ProductID;
```

Concept: PARTITION BY

---

38. Display Month-over-Month Sales Growth

```
WITH MonthlySales AS
(
   SELECT
      YEAR(PaymentDate) AS Yr,
      MONTH(PaymentDate) AS Mn,
      SUM(AmountPaid) AS Revenue
   FROM payments
   GROUP BY
      YEAR(PaymentDate),
      MONTH(PaymentDate)
)

SELECT

Yr,

Mn,

Revenue,

LAG(Revenue)
OVER
(
ORDER BY Yr,Mn
) PreviousRevenue

FROM MonthlySales;
```

Concept: LAG()

---

39. Categorize Customers Based on Revenue

```
SELECT

CustomerID,

CustomerName,

Revenue,

CASE
```

```
WHEN Revenue > 100000 THEN 'Platinum'

WHEN Revenue > 50000 THEN 'Gold'

WHEN Revenue > 20000 THEN 'Silver'

ELSE 'Bronze'

END AS CustomerCategory

FROM
(
SELECT

c.CustomerID,

c.CustomerName,

SUM(p.AmountPaid) AS Revenue

FROM customers c

INNER JOIN orders o

ON c.CustomerID = o.CustomerID

INNER JOIN payments p

ON o.OrderID = p.OrderID

GROUP BY
c.CustomerID,
c.CustomerName

) x;
```

Concept: CASE

---

40. Create Stored Procedure to Get Customer Orders

```
DROP PROCEDURE IF EXISTS GetCustomerOrders;

DELIMITER //

CREATE PROCEDURE GetCustomerOrders
(
   IN p_customerid INT
)
BEGIN

SELECT *
```

```
FROM orders
WHERE CustomerID = p_customerid;
END //
DELIMITER ;
CALL GetCustomerOrders(101);
```

Concept: Stored Procedure (Input Parameter)

---

41. Create Stored Procedure to Display Monthly Sales

```
DROP PROCEDURE IF EXISTS MonthlySalesReport;
DELIMITER //
CREATE PROCEDURE MonthlySalesReport()
BEGIN
SELECT
YEAR(PaymentDate) YearNo,
MONTH(PaymentDate) MonthNo,
SUM(AmountPaid) Revenue
FROM payments
GROUP BY
YEAR(PaymentDate),
MONTH(PaymentDate);
END //
DELIMITER ;
CALL MonthlySalesReport();
```

Concept: Stored Procedure

---

42. Create Stored Procedure to Display Top 10 Customers

```
DROP PROCEDURE IF EXISTS TopCustomers;
```

```
DELIMITER //
CREATE PROCEDURE TopCustomers()
BEGIN
SELECT
c.CustomerID,
c.CustomerName,
SUM(p.AmountPaid) Revenue
FROM customers c
INNER JOIN orders o
ON c.CustomerID=o.CustomerID
INNER JOIN payments p
ON o.OrderID=p.OrderID
GROUP BY
c.CustomerID,
c.CustomerName
ORDER BY Revenue DESC
LIMIT 10;
END //
DELIMITER ;
CALL TopCustomers();
```

Concept: Stored Procedure

---

43. Create BEFORE INSERT Trigger to Validate Payment

```
DELIMITER //
CREATE TRIGGER ValidatePayment
BEFORE INSERT
ON payments
```

```
FOR EACH ROW

BEGIN

IF NEW.AmountPaid <=0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Payment amount must be greater than zero';

END IF;

END //

DELIMITER ;
```

Concept: BEFORE INSERT Trigger

---

44. Create AFTER INSERT Trigger to Update Inventory

```
DELIMITER //

CREATE TRIGGER UpdateInventory

AFTER INSERT

ON orderdetails

FOR EACH ROW

BEGIN

UPDATE inventory

SET Stock = Stock - NEW.Quantity

WHERE ProductID = NEW.ProductID;

END //

DELIMITER ;
```

Concept: AFTER INSERT Trigger

---

45. Create BEFORE DELETE Trigger to Prevent Customer Deletion

```
DELIMITER //

CREATE TRIGGER PreventCustomerDelete
```

```
BEFORE DELETE

ON customers

FOR EACH ROW

BEGIN

IF EXISTS
(
SELECT 1
FROM orders
WHERE CustomerID=OLD.CustomerID
)

THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Customer has orders. Deletion not allowed.';

END IF;

END //

DELIMITER ;
```

Concept: BEFORE DELETE Trigger